Distributed ML with CycleCloud & NGC

2021. 05. 28.

Sooyoung Moon

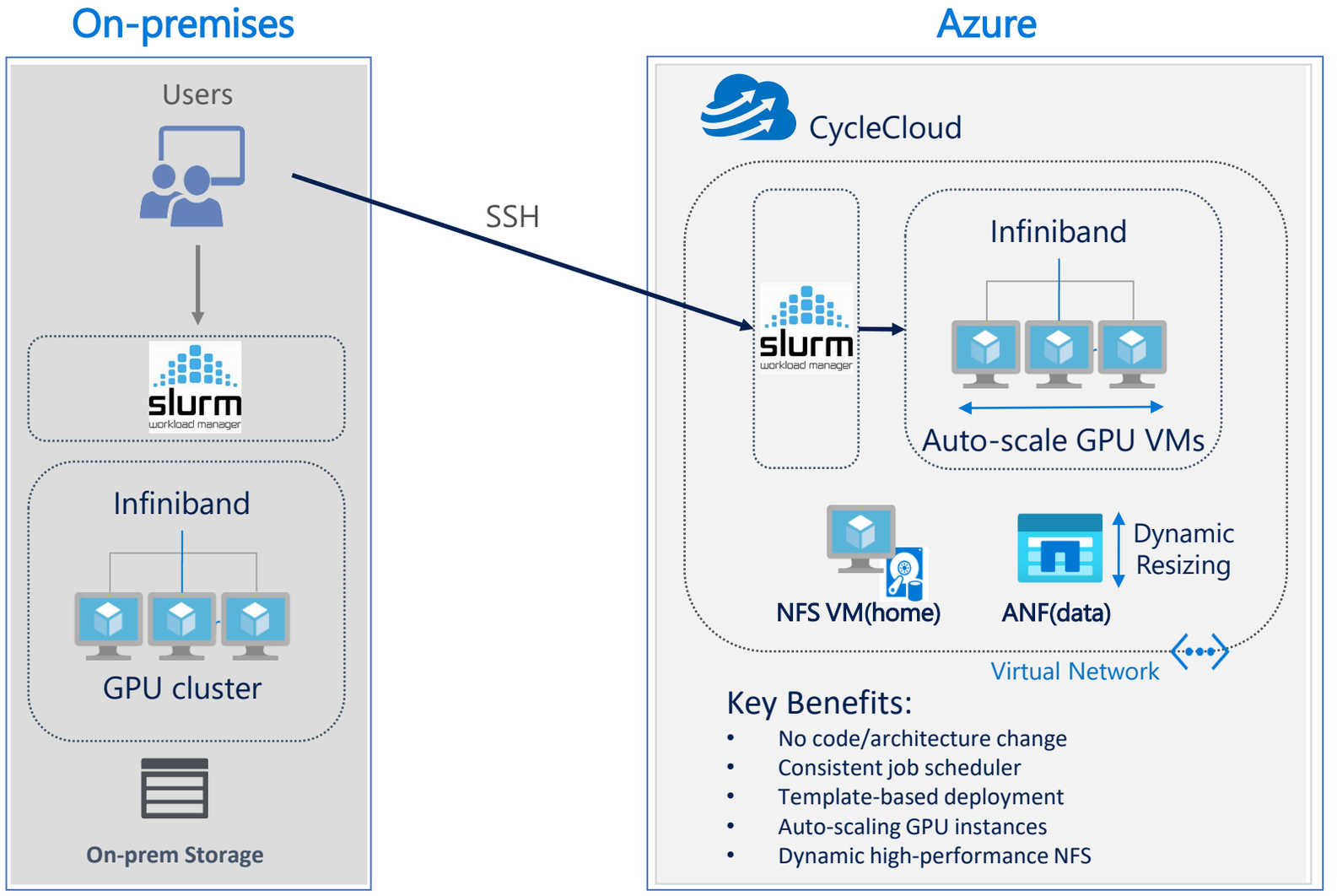
Microsoft HPC Global Blackbelt

Contributors

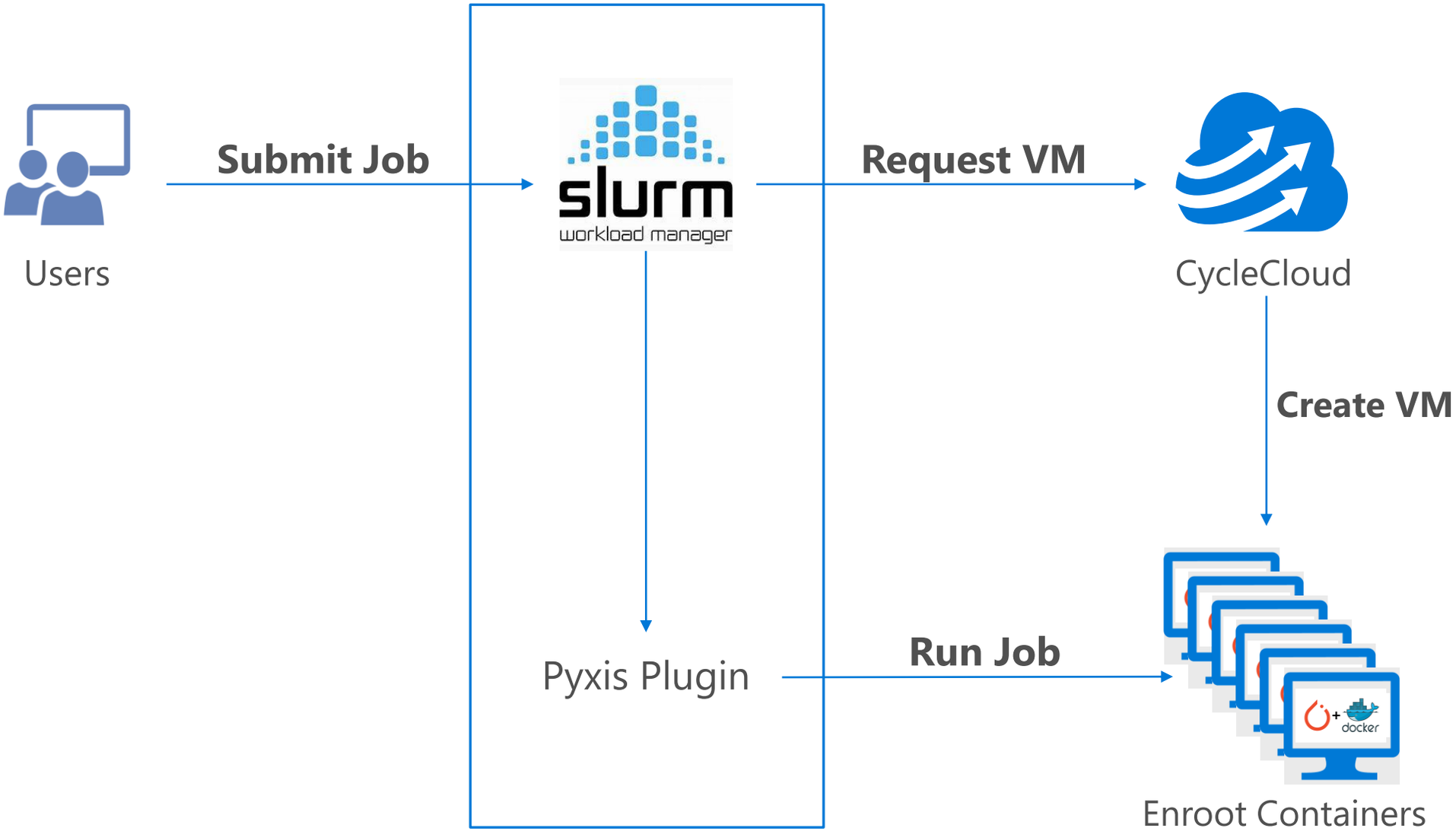
- Alex Sutton
- Aman Verma
- Andy Howard
- Ben Watrous
- Carl Chesal
- Cormac Garvey
- Dan Harris
- David Feng
- Eric ZHAO
- Eunyoung Oh
- Ian Finder
- Ignacio Martinez
- Ihyun Cho
- Jer-Ming Chia
- Jerry Morey
- Jithin Jose
- Ji Zhou
- Jon Shelley
- Kanchan Mehrotra
- Ke Zhao
- Mohammad Siddiqui
- Naoyuki Isogai
- Rob Futrick
- Sam Kemp
- Seokjin Han
- Simran Amora
- Shawn Liu
- Timo Klimmer
- Vinil Vadakkepurakkal
- Vivek Ramaswamy

And more who I cannot remember now because of my poor memory for names. ☺

Reference Architecture: Distributed ML with CycleCloud



CycleCloud integration with NGC containers



Key Components (example)

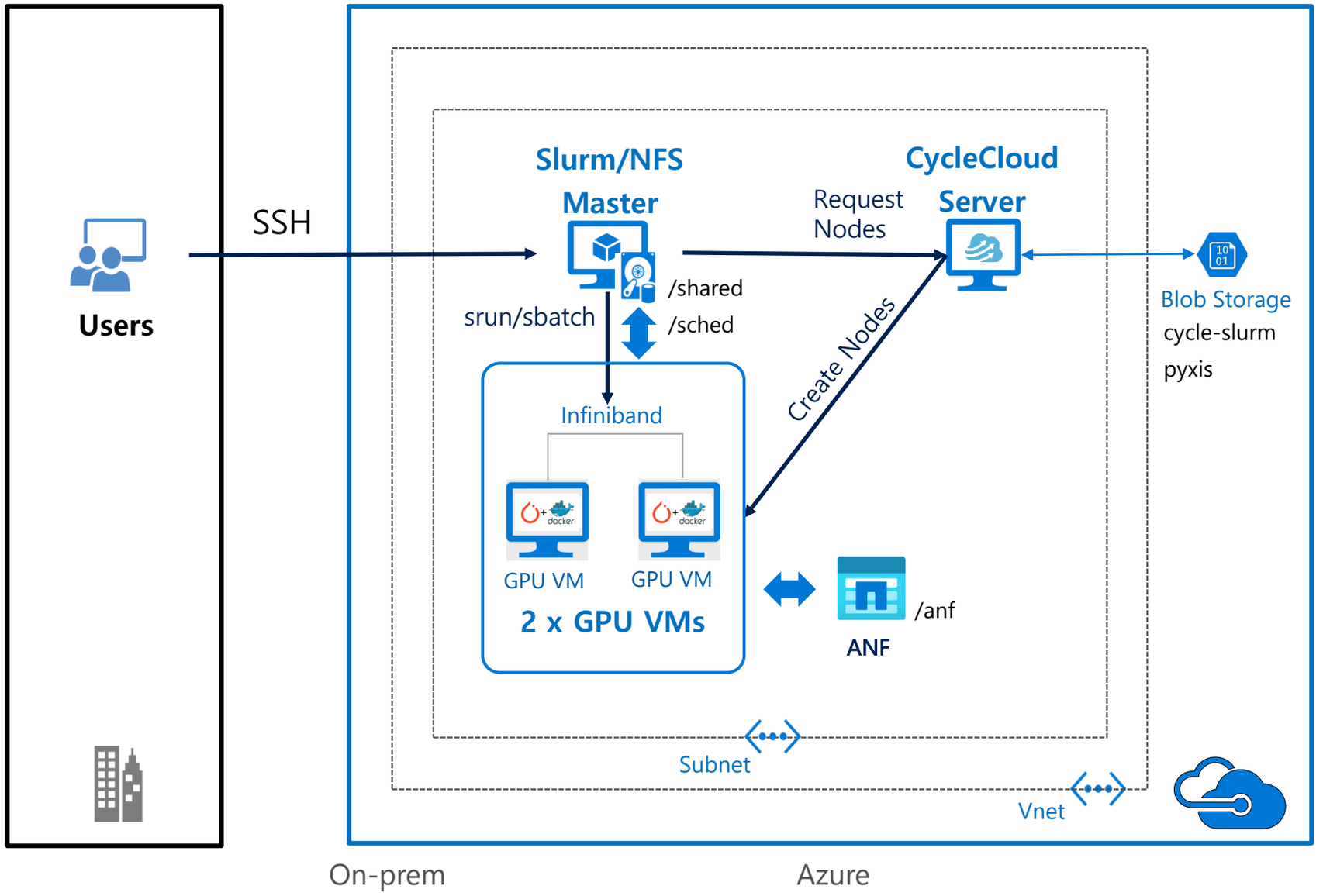
VM/Cluster

1. GPU VMs (V100 or A100)
2. Ubuntu 18.04 or later
3. Nvidia driver 450.80.02 or later
4. Topology file (ND_v4 only)
5. Slurm 20.02 or later
6. Enroot 3.1.0 or later
7. Pyxis 0.9.1 or later
8. CycleCloud Slurm project 2.4.4 or later
9. CycleCloud 8.1.0 or later

Container

1. NGC PyTorch Container 20.12
   - Ubuntu 20.04
   - Python 3.8
   - CUDA 11.1.1
   - NCCL 2.8.3
   - APEX
   - MLNX OFED
   - UCX 1.9.0
   - OpenMPI 4.0.5
   - PyTorch 1.8
2. nccl rdma sharp plugin

A Lab Configuration



Prepare the custom image

1. Start a GPU VM with Ubuntu 18.04
2. Log in to the VM
3. Install Nvidia driver

```
$ sudo apt install -y nvidia-driver-450
```

4. Create a custom image

```
$ sudo waagent -deprovision+user
$ exit
```

5. From Azure Cloudshell,

```
$ az vm deallocate -n <vm name> -g <group name>
$ az vm generalize -n <vm name> -g <group name>
$ az image create -n <image name> --source <vm name> -g <group name>
#NOTE: In case of Gen2 VM, append "--hyper-v-generation V2".
```

Prepare the Enroot project(1/3)

Start a CycleCloud server

Log into the CycleCloud server

$ cyclecloud project init enroot

$ cd enroot/specs/default/cluster-init/scripts

$ vi 01-enroot-setup.sh

```
#!/bin/bash

set -ex

# Install Enroot from packages
# Ref: https://github.com/NVIDIA/enroot/blob/master/doc/installation.md
# For the latest version, refer to https://github.com/NVIDIA/enroot/blob/master/doc/installation.md#standard-flavor
apt update
apt install -y jq parallel zstd pigz
arch=$(dpkg --print-architecture)
curl -fSsL -O https://github.com/NVIDIA/enroot/releases/download/v3.3.0/enroot_3.3.0-1_${arch}.deb
curl -fSsL -O https://github.com/NVIDIA/enroot/releases/download/v3.3.0/enroot+caps_3.3.0-1_${arch}.deb
apt install -y ./enroot_3.3.0-1_${arch}.deb ./enroot+caps_3.3.0-1_${arch}.deb
enroot version
```

Prepare the Enroot project (2/3)

```
# Configure Enroot runtime directories
# Ref: https://github.com/NVIDIA/enroot/blob/master/doc/configuration.md
mkdir -m 1777 -p /mnt/enroot
echo "@reboot mkdir -m 1777 -p /mnt/enroot" | crontab -
sudo crontab -l

cat << END >> /etc/enroot/enroot.conf
ENROOT_RUNTIME_PATH     /mnt/enroot/\$UID/run          # Default: /run/user/$UID/enroot
ENROOT_CONFIG_PATH      \$HOME/.config/enroot          # Default: $HOME/.config/enroot
ENROOT_CACHE_PATH       /mnt/enroot/\$UID/.cache       # Default: $HOME/.cache/enroot
ENROOT_DATA_PATH        /mnt/enroot/\$UID/.data        # Default: $HOME/.local/share/enroot
ENROOT_TEMP_PATH        /mnt/enroot                    # Default: /tmp
END

# Install Pyxis from source
# Ref: https://github.com/NVIDIA/pyxis/wiki/Installation
# Slurm needs to be installed and started before running this script.
apt install -y gcc make
git clone https://github.com/NVIDIA/pyxis.git
cd pyxis
make install
ln -s /usr/local/share/pyxis/pyxis.conf  /etc/slurm/plugstack.conf
systemctl restart slurmd
```

Prepare the Enroot project (3/3)

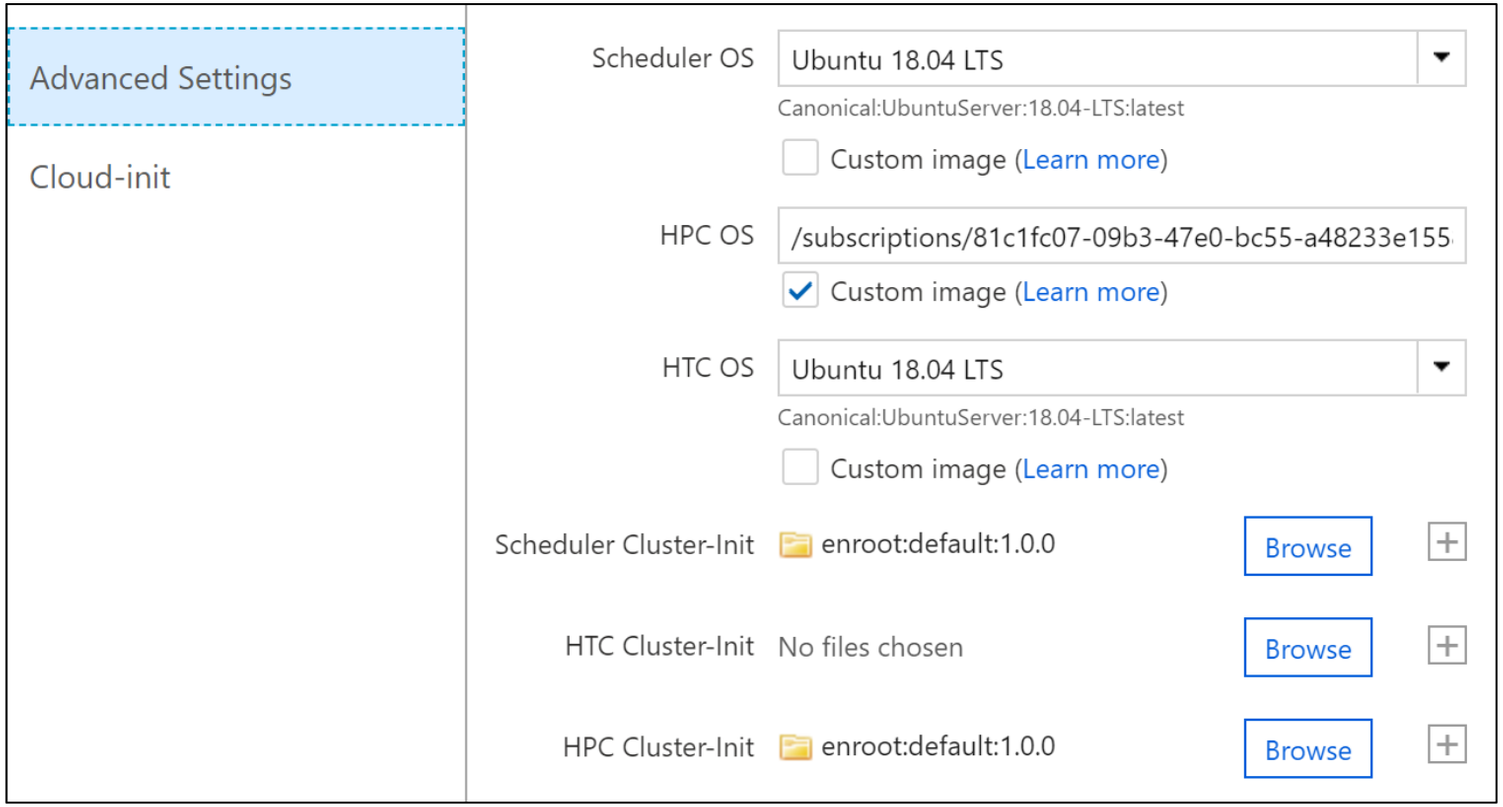
```
# Install Nvidia container tools(Required to run Enroot)
# Ref: https://github.com/NVIDIA/libnvidia-container
DIST=$(. /etc/os-release; echo $ID$VERSION_ID)
curl -s -L https://nvidia.github.io/libnvidia-container/gpgkey | apt-key add -
curl -s -L https://nvidia.github.io/libnvidia-container/$DIST/libnvidia-container.list | tee /etc/apt/sources.list.d/libnvidia-container.list
apt update
apt install -y libnvidia-container-tools
```

```
$ cyclecloud initialize  # You can use https://localhost for the URL.
$ storage=`cyclecloud locker list | cut -d " " -f1`
$ cyclecloud project upload $storage
```

Start the cluster

In Cycle UI:

- Slurm Version: 20.11.4-1
- Scheduler OS: Ubuntu 18.04 LTS
- HPC OS: The custom image resource ID
- HTC OS: Ubuntu 18.04 LTS
- Scheduler Cluster-init: enroot:default:1.0.0
- HPC Cluster-init: enroot:default:1.0.0



Once started, login to the scheduler VM.

Enroot exercises (1/2)

#Ref: https://github.com/NVIDIA/enroot/blob/master/doc/usage.md

1. Check ENROOT_CONFIG_PATH in enroot.conf

$ cat /etc/enroot/enroot.conf

2. Configure container credentials

#Ref: https://github.com/NVIDIA/enroot/blob/master/doc/cmd/import.md#description
#Ref: https://ngc.nvidia.com/setup/api-key
$ mkdir -p $HOME/.config/enroot
$ vi $HOME/.config/enroot/.credentials
machine nvcr.io login $oauthtoken password <NGC API Key>

3. Import an container image from NGC repository

cd /mnt

enroot import docker://nvcr.io/nvidia/pytorch:20.12-py3

'enroot import docker://$oauthtoken@nvcr.io#nvidia/pytorch:20.12-py3' in case of no NGC API Key.

ls

enroot create -n fun-pytorch nvidia/pytorch:20.12-py3.sqsh

enroot list

Enroot exercises (2/2)

4. Import an image from local Docker

$ docker pull centos

$ docker images

$ exit

$ enroot import -o centos.sqsh dockerd://centos:latest

$ enroot create -n fun-centos ./centos.sqsh

$ enroot list

5. Create a custom Enroot image

$ enroot start fun-centos ls /workspace

$ enroot start -w fun-centos

$ touch iwashere

$ exit

$ enroot export -o custom-centos.sqsh fun-centos

$ enroot create -n custom-centos ./custom-centos.sqsh

$ enroot start custom-centos ls /workspace

Install Nvidia Docker to set up BERT

1. Start a GPU VM (Temporary)

```
$ salloc -w hpc-pg0-1 --gpus=1
```

2. Setting up Docker

```
# Ref: https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/install-guide.html#docker
$ ssh <GPU VM IP>.
$ sudo mkdir /mnt/docker
$ sudo ln -s /mnt/docker /var/lib/docker
$ curl https://get.docker.com | sh && sudo systemctl --now enable docker
$ sudo docker info
$ sudo usermod -a -G docker $USER
$ exit  # Log out from the compute node.
```

3. Setting up Nvidia Container Toolkit

```
$ srun --pty bash #  Or, SSH into the node.
$ distribution=$(. /etc/os-release;echo $ID$VERSION_ID) \
   && curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey | sudo apt-key add - \
   && curl -s -L https://nvidia.github.io/nvidia-docker/$distribution/nvidia-docker.list | sudo tee /etc/apt/sources.list.d/nvidia-docker.list
$ sudo apt-get update
$ sudo apt-get install -y nvidia-docker2
$ sudo systemctl restart docker
$ docker run --rm --gpus all nvidia/cuda:11.0-base nvidia-smi # To check if Docker runs fine.
```

Set up BERT

#Ref: https://github.com/NVIDIA/DeepLearningExamples/tree/master/PyTorch/LanguageModeling/BERT#quick-start-guide

1. Clone the repository

$ cd /anf
$ git clone https://github.com/NVIDIA/DeepLearningExamples.git
$ mv ./DeepLearningExamples/PyTorch/LanguageModeling/BERT .
$ cd BERT

2. Download the Nvidia pretrained checkpoint (optional)

# Ref: https://ngc.nvidia.com/catalog/models/nvidia:bert_pyt_ckpt_large_pretraining_amp_lamb/files
$ sudo apt install -y unzip
$ wget --content-disposition https://api.ngc.nvidia.com/v2/models/nvidia/bert_pyt_ckpt_large_pretraining_amp_lamb/versions/19.07.0/zip \
-O bert_pyt_ckpt_large_pretraining_amp_lamb_19.07.0.zip
$ unzip bert_pyt_ckpt_large_pretraining_amp_lamb_19.07.0.zip
$ mv DLE_BERT_FP16_PyT_LAMB_92_hard_scaling_node.pt checkpoints/

3. Build BERT on top of the NGC container.

$ bash scripts/docker/build.sh # This takes about 10min.
$ docker images

Prepare BERT dataset

1. Start an interactive session in the NGC container

$ bash scripts/docker/launch.sh

```
#!/bin/bash

CMD=${1:-/bin/bash}
NV_VISIBLE_DEVICES=${2:-"all"}
DOCKER_BRIDGE=${3:-"host"}

docker run -it --rm \
  --gpus device=$NV_VISIBLE_DEVICES \
  --net=$DOCKER_BRIDGE \
  --shm-size=1g \
  --ulimit memlock=-1 \
  --ulimit stack=67108864 \
  -e LD_LIBRARY_PATH='/workspace/install/lib/' \
  -v $PWD:/workspace/bert \
  -v $PWD/results:/results \
  bert $CMD
```

Prepare BERT dataset (continued)

2. Download and preprocess the dataset

$ data/create_datasets_from_start.sh wiki_only # Takes ~15 hours and ~150GB disk spae.

```
#!/bin/bash
to_download=${1:-"wiki_only"}

#Download
if [ "$to_download" = "wiki_books" ] ; then
    python3 /workspace/bert/data/bertPrep.py --action download --dataset bookscorpus
fi

python3 /workspace/bert/data/bertPrep.py --action download --dataset wikicorpus_en
python3 /workspace/bert/data/bertPrep.py --action download --dataset google_pretrained_weights  # Includes vocab
python3 /workspace/bert/data/bertPrep.py --action download --dataset squad
python3 /workspace/bert/data/bertPrep.py --action download --dataset mrpc
python3 /workspace/bert/data/bertPrep.py --action download --dataset sst-2

# Properly format the text files
if [ "$to_download" = "wiki_books" ] ; then
    python3 /workspace/bert/data/bertPrep.py --action text_formatting --dataset bookscorpus
fi
python3 /workspace/bert/data/bertPrep.py --action text_formatting --dataset wikicorpus_en

if [ "$to_download" = "wiki_books" ] ; then
    DATASET="books_wiki_en_corpus"
else
    DATASET="wikicorpus_en"
    # Shard the text files
fi

# Shard the text files
python3 /workspace/bert/data/bertPrep.py --action sharding --dataset $DATASET

# Create HDF5 files Phase 1
python3 /workspace/bert/data/bertPrep.py --action create_hdf5_files --dataset $DATASET --max_seq_length 128 \
--max_predictions_per_seq 20 --vocab_file $BERT_PREP_WORKING_DIR/download/google_pretrained_weights/uncased_L-24_H-1024_A-16/vocab.txt --do_lower_case 1

# Create HDF5 files Phase 2
python3 /workspace/bert/data/bertPrep.py --action create_hdf5_files --dataset $DATASET --max_seq_length 512 \
--max_predictions_per_seq 80 --vocab_file $BERT_PREP_WORKING_DIR/download/google_pretrained_weights/uncased_L-24_H-1024_A-16/vocab.txt --do_lower_case 1
```

Don't download the raw data. Instead, use the preprocessed data shared before this lab.

Prepare BERT job scripts

 # Prepare the job script

$ vi ./bind.sh
Fix the typo below:
#! /bin/bash -> #!/bind.sh (No space is allowed!)

chmod +x bind.sh

$ vi ./run.sub
Need to change these variable values as appropriate:
#SBATCH --nodes=2
#SBATCH --gpus-per-node=2
#SBATCH --ntasks-per-node=2
#SBATCH --partition=hpc

export MASTER_ADDR=`hostname -I | cut -d' ' -f1`
export MASTER_PORT=`comm -23 <(seq 49152 65535 | sort) <(ss -Htan | awk '{print $4}' | cut -d':' -f2 | sort -u) | shuf | head -n 1`
export WORLD_SIZE=$((SLURM_JOB_NUM_NODES * SLURM_GPUS_PER_NODE))

Run BERT pre-training

1. Submit the job

$ BATCHSIZE=2048 LR=6e-3 GRADIENT_STEPS=128 PHASE=1 sbatch run.sub #Need to change run.sub appropriately.

```
#!/bin/bash
#SBATCH --exclusive
#SBATCH --mem=0
#SBATCH --overcommit

# The following variables need to be set
# Base container to be used - container built in step 1 on quick start guide
readonly docker_image="nvcr.io/nvidia/pytorch:20.06-py3"
# Location of dataset for phase 1
readonly datadir=" /anf/BERT/data/hdf5_lower_case_1_seq_len_128_max_pred_20_masked_lm_prob_0.15_random_seed_12345_dupe_factor_5/wikicorpus_en "
# Location of dataset for phase 2
readonly datadir_phase2="/anf/BERT/data/hdf5_lower_case_1_seq_len_512_max_pred_80_masked_lm_prob_0.15_random_seed_12345_dupe_factor_5/wikicorpus_en"
# Path to where trained checkpoints will be saved on the system
readonly checkpointdir="$PWD/checkpoints"

BERT_CMD="\
    ${BIND_CMD} python -u /workspace/bert/run_pretraining.py \
    --seed=42 \
    ${PHASES[$((PHASE-1))]} \
    --do_train \
    --config_file=/workspace/bert/bert_config.json \
    --output_dir=/results \
    --fp16 \
    --allreduce_post_accumulation --allreduce_post_accumulation_fp16 \
    --gradient_accumulation_steps=${GRADIENT_STEPS:-2} \
    --log_freq=1 \
    --local_rank=\${SLURM_LOCALID}"

srun -l --container-image="${docker_image}" --container-mounts="${mounts}" --container-workdir="/workspace/bert" sh -c "${BERT_CMD}"
```

Run BERT pre-training (continued)

Monitor Slurm output

$ tail -f slurm_<job id>.out

```
+ export MASTER_PORT=53849
+ MASTER_PORT=53849
+ export WORLD_SIZE=8
+ WORLD_SIZE=8
+ export NCCL_DEBUG=INFO
+ NCCL_DEBUG=INFO
+ BIND_CMD='./bind.sh '
+ srun --ntasks=2 --ntasks-per-node=1 mkdir -p /anf/BERT/checkpoints
+ PHASE1='    --train_batch_size=16     --learning_rate=6e-3     --warmup_proportion=0.2843     --input_dir=/workspace/data     --max_seq_length=128     --max_prediction
s_per_seq=20     --max_steps=7038     --num_steps_per_checkpoint=2500     '
+ PHASE2='    --train_batch_size=4096     --learning_rate=4e-3     --warmup_proportion=0.128     --input_dir=/workspace/data_phase2     --phase2     --max_seq_length=512
     --max_predictions_per_seq=80     --max_steps=1563     --num_steps_per_checkpoint=1000     --resume_from_checkpoint --phase1_end_step=7038     '
+ PHASES=("$PHASE1" "$PHASE2")
+ PHASE=1
+ BERT_CMD='    ./bind.sh  python -u /workspace/bert/run_pretraining.py     --seed=42         --train_batch_size=16     --learning_rate=6e-3     --warmup_proportion=0.28
43     --input_dir=/workspace/data     --max_seq_length=128     --max_predictions_per_seq=20     --max_steps=7038     --num_steps_per_checkpoint=2500         --do_train
     --config_file=/workspace/bert/bert_config.json     --output_dir=/results     --fp16     --allreduce_post_accumulation --allreduce_post_accumulation_fp16     --gradi
ent_accumulation_steps=2     --log_freq=1     --local_rank=${SLURM_LOCALID}'
+ srun -l --container-image=nvcr.io/nvidia/pytorch:20.06-py3 --container-mounts=/anf/BERT:/workspace/bert,/anf/BERT/data/hdf5_lower_case_1_seq_len_128_max_pred_20_masked
_lm_prob_0.15_random_seed_12345_dupe_factor_5/wikicorpus_en:/workspace/data,/anf/BERT/data/hdf5_lower_case_1_seq_len_512_max_pred_80_masked_lm_prob_0.15_random_seed_1234
5_dupe_factor_5/wikicorpus_en:/workspace/data_phase2,/anf/BERT/checkpoints:/results --container-workdir=/workspace/bert sh -c '    ./bind.sh  python -u /workspace/bert/r
un_pretraining.py     --seed=42         --train_batch_size=16     --learning_rate=6e-3     --warmup_proportion=0.2843     --input_dir=/workspace/data     --max_seq_lengt
h=128     --max_predictions_per_seq=20     --max_steps=7038     --num_steps_per_checkpoint=2500         --do_train     --config_file=/workspace/bert/bert_config.json
  --output_dir=/results     --fp16     --allreduce_post_accumulation --allreduce_post_accumulation_fp16     --gradient_accumulation_steps=2     --log_freq=1     --local_
rank=${SLURM_LOCALID}'
1: pyxis: importing docker image ...
7: pyxis: importing docker image ...
[4: num_sockets = 2 num_nodes=2 cores_per_socket=12
5: num_sockets = 2 num_nodes=2 cores_per_socket=12
4: + exec numactl --cpunodebind=0 --membind=0 -- python -u /workspace/bert/run_pretraining.py --seed=42 --train_batch_size=16 --learning_rate=6e-3 --warmup_proportion=0.
2843 --input_dir=/workspace/data --max_seq_length=128 --max_predictions_per_seq=20 --max_steps=7038 --num_steps_per_checkpoint=2500 --do_train --config_file=/workspace/b
ert/bert_config.json --output_dir=/results --fp16 --allreduce_post_accumulation --allreduce_post_accumulation_fp16 --gradient_accumulation_steps=2 --log_freq=1 --local_r
ank=0
```

Run BERT pre-training (continued)

You did it successfully!


```
5: device: cuda:1 n_gpu: 1, distributed training: True, 16-bits training: True
6: device: cuda:2 n_gpu: 1, distributed training: True, 16-bits training: True
7: device: cuda:3 n_gpu: 1, distributed training: True, 16-bits training: True
4: device: cuda:0 n_gpu: 1, distributed training: True, 16-bits training: True
1: device: cuda:1 n_gpu: 1, distributed training: True, 16-bits training: True
2: device: cuda:2 n_gpu: 1, distributed training: True, 16-bits training: True
3: device: cuda:3 n_gpu: 1, distributed training: True, 16-bits training: True
0: device: cuda:0 n_gpu: 1, distributed training: True, 16-bits training: True
```

```
0: DLL 2021-05-27 12:01:50.405920 - PARAMETER loss_scale : 65536.0
0: DLL 2021-05-27 12:01:50.409419 - Training Epoch: 0 Training Iteration: 0  average_loss : 11.18359375  step_loss : 11.1171875  learning_rate : 2.9986455118223097e-06
0: DLL 2021-05-27 12:01:54.098207 - PARAMETER loss_scale : 32768.0
0: DLL 2021-05-27 12:01:54.101719 - Training Epoch: 0 Training Iteration: 0  average_loss : 11.1015625  step_loss : 11.2421875  learning_rate : 2.9986455118223097e-06
0: DLL 2021-05-27 12:01:58.107743 - Training Epoch: 0 Training Iteration: 1  average_loss : 11.296875  step_loss : 11.3515625  learning_rate : 2.9986455118223097e-06
0: DLL 2021-05-27 12:02:02.165944 - Training Epoch: 0 Training Iteration: 2  average_loss : 11.2109375  step_loss : 10.9609375  learning_rate : 5.997291023644619e-06
0: DLL 2021-05-27 12:02:05.887435 - Training Epoch: 0 Training Iteration: 3  average_loss : 11.20703125  step_loss : 11.25  learning_rate : 8.995936535466931e-06
0: DLL 2021-05-27 12:02:09.634445 - Training Epoch: 0 Training Iteration: 4  average_loss : 11.25390625  step_loss : 11.1953125  learning_rate : 1.1994582047289239e-05
0: DLL 2021-05-27 12:02:13.341535 - Training Epoch: 0 Training Iteration: 5  average_loss : 11.265625  step_loss : 11.2890625  learning_rate : 1.499322755911155e-05
0: DLL 2021-05-27 12:02:17.056080 - Training Epoch: 0 Training Iteration: 6  average_loss : 11.2578125  step_loss : 11.1796875  learning_rate : 1.7991873070933862e-05
0: DLL 2021-05-27 12:02:20.821112 - Training Epoch: 0 Training Iteration: 7  average_loss : 11.15625  step_loss : 11.203125  learning_rate : 2.099051858275617e-05
0: DLL 2021-05-27 12:02:24.591690 - Training Epoch: 0 Training Iteration: 8  average_loss : 11.20703125  step_loss : 11.1875  learning_rate : 2.3989164094578478e-05
0: DLL 2021-05-27 12:02:28.494059 - Training Epoch: 0 Training Iteration: 9  average_loss : 11.0390625  step_loss : 11.0234375  learning_rate : 2.698780960640079e-05
Iteration:   0%|          | 26/81900 [00:50<42:54:14,  1.89s/it]
0: DLL 2021-05-27 12:02:32.512277 - Training Epoch: 0 Training Iteration: 10  average_loss : 11.1640625  step_loss : 11.109375  learning_rate : 2.99864551182231e-05
0: DLL 2021-05-27 12:02:36.401322 - Training Epoch: 0 Training Iteration: 11  average_loss : 10.88671875  step_loss : 10.9140625  learning_rate : 3.298510063004541e-05
0: DLL 2021-05-27 12:02:40.211151 - Training Epoch: 0 Training Iteration: 12  average_loss : 10.94140625  step_loss : 10.9765625  learning_rate : 3.5983746141867724e-05
0: DLL 2021-05-27 12:02:44.127793 - Training Epoch: 0 Training Iteration: 13  average_loss : 10.984375  step_loss : 10.984375  learning_rate : 3.898239165369003e-05
0: DLL 2021-05-27 12:02:47.949200 - Training Epoch: 0 Training Iteration: 14  average_loss : 10.75390625  step_loss : 10.875  learning_rate : 4.198103716551234e-05
0: DLL 2021-05-27 12:02:51.907537 - Training Epoch: 0 Training Iteration: 15  average_loss : 10.7890625  step_loss : 10.703125  learning_rate : 4.497968267733465e-05
```

References

1. Instructions for building a Ubuntu 18.04 image for Azure ND_v4 VM (github.com)
2. Scheduling with SLURM and Enroot(GTC 2021)
3. Slurm integration with Unprivileged Containers
4. CycleCloud/Slurm Scheduler Integration
5. Azure/cyclecloud-slurm project
6. NVIDIA/nvidia-docker: Build and run Docker containers leveraging NVIDIA GPUs (github.com)
7. NVIDIA Enroot
8. NVIDIA Pyxis
9. NVIDIA Container Support Matrix
10. NVIDIA/nccl: Optimized primitives for collective multi-GPU communication (github.com)
11. NVIDIA Collective Communications Library (NCCL) | NVIDIA Developer
12. NVIDIA Collective Communication Library (NCCL) Documentation
13. Mellanox SHARP with NVIDIA NCCL
14. Mellanox/nccl-rdma-sharp-plugins: RDMA and SHARP plugins for nccl library (github.com)
15. PyTorch | NVIDIA NGC
16. NVIDIA Deep Learning Examples (github.com)
17. NVIDIA/Megatron-LM

Thank You

Appendix: GPT-3 slurm script example

```
#!/bin/bash
#SBATCH --job-name=test
#SBATCH --nodes=2
#SBATCH --gpus-per-node=8
#SBATCH --ntasks-per-node=8
#SBATCH --output=logs/%j.%x.info.log
#SBATCH --error=logs/%j.%x.error.log
...
export WORLD_SIZE=$((SLURM_JOB_NUM_NODES * SLURM_GPUS_PER_NODE))
ENROOT_IMAGE_PATH=/cycleanf/xxxxxx
DATASET_0=xxx
DATASET_1=xxx
...
export NCCL_DEBUG=INFO
export UCX_IB_ENABLE_CUDA_AFFINITY=n
export UCX_TLS=rc
export UCX_NET_DEVICES=ibP257p0s0:1,ibP258p0s0:1,ibP259p0s0:1,ibP260p0s0:1,ibP261p0s0:1,ibP262p0s0:1,ibP263p0s0:1,ibP264p0s0:1
export CUDA_DEVICE_ORDER=PCI_BUS_ID
export NCCL_IB_PCI_RELAXED_ORDERING=1
export NCCL_TOPO_FILE=/cycleanf/topo.xml
export OMPI_MCA_pml=ucx
export OMPI_MCA_btl=^openib
```

Appendix: GPT-3 slurm script example (continued)

```
export LD_LIBRARY_PATH=/usr/local/nccl-rdma-sharp-plugins/lib:$LD_LIBRARY_PATH
export NCCL_PLUGIN_P2P=ucx

run_cmd="unset CXX; unset CC; python3 -u ${DIR}/pretrain_gpt.py $@ ${options} "

srun -l \
--container-image="$ENROOT_IMAGE_PATH/custom_image.sqsh" \
--container-mounts="/cycleanf:/cycleanf" \
--container-name= "custom_container" \
--output=$LOG_PATH/%j.%x.log sh -c "$run_cmd"

set +x

# NOTE: The default P2P with plugin is native NCCL IB (NCCL_PLUGIN_P2P=Ib). NCCL_PLUGIN_P2P=ucx is required to force UCX.
```

Appendix: Important Slurm command options for Enroot

--container-image=[USER@][REGISTRY#]IMAGE[:TAG]|PATH

- The image to use for the container filesystem. Can be either a docker image given as an enroot URI, or a path to a squashfs file on the remote host filesystem.

--container-mounts=SRC:DST[:FLAGS][,SRC:DST...]

- Bind mount[s] inside the container. Mount flags are separated with "+", e.g. "ro+rprivate"

--container-workdir=PATH

- Working directory inside the container

--container-name=NAME

- Name to use for saving and loading the container on the host. Unnamed containers are removed after the slurm task is complete; named containers are not. If a container with this name already exists, the existing container is used and the import is skipped.

Appendix: How to install Enroot from source

```
#!/bin/bash
# Ref: https://github.com/NVIDIA/enroot/blob/master/doc/installation.md
# Install the build dependencies
set -ex
apt update
apt install -y git gcc make libcap2-bin libtool automake
#Install the runtime dependencies:
apt install -y curl gawk jq squashfs-tools parallel

# Install fuse-overlayfs # NOTE: With Ubuntu 20.04 or later, you can install fuse-overlayfs by running ‘apt update && apt install fuse-overlayfs’ instead.
# Install buildah (Prerequisite to fuse-overlayfs)
# Ref: https://github.com/containers/buildah/blob/master/install.md
apt install -y software-properties-common
add-apt-repository -y ppa:alexlarsson/flatpak
add-apt-repository -y ppa:gophers/archive
apt-add-repository -y ppa:projectatomic/ppa
apt install -y bats btrfs-tools git libapparmor-dev libdevmapper-dev libglib2.0-dev libgpgme11-dev libseccomp-dev libselinux1-dev skopeo-containers go-md2man
apt install -y golang-1.13 buildah
# Install fuse-overlayfs from source
# Ref: https://github.com/containers/fuse-overlayfs
cd /mnt
git clone https://github.com/containers/fuse-overlayfs.git
cd fuse-overlayfs
buildah bud -v $PWD:/build/fuse-overlayfs -t fuse-overlayfs -f ./Containerfile.static.ubuntu . # NOTE: Make sure to include '.' at the end.
cp fuse-overlayfs /usr/bin/
```

Appendix: How to install Enroot from source(continued)

```
# Install Nvidia container tools library
# Ref: https://github.com/NVIDIA/libnvidia-container
DIST=$(. /etc/os-release; echo $ID$VERSION_ID)
curl -s -L https://nvidia.github.io/libnvidia-container/gpgkey | apt-key add -
curl -s -L https://nvidia.github.io/libnvidia-container/$DIST/libnvidia-container.list | tee /etc/apt/sources.list.d/libnvidia-container.list
apt-get update
apt install -y libnvidia-container1 libnvidia-container-tools
apt install -y pigz squashfuse (optional)

# Build and install Enroot
cd /mnt
git clone --recurse-submodules https://github.com/NVIDIA/enroot.git
cd enroot
make install
# Allow unprivileged users to import images:
make setcap
```

Appendix: How to install Pyxis from source

```
$ apt install devscripts debhelper
$ cd pyxis/debian
$ vi control
libslurm-dev -> slurm-devel
$ cd ..
$ make orig
$ make deb
$ sudo dpkg -i ../nvslurm-plugin-pyxis_*_amd64.deb
$ sudo ln -s /usr/share/pyxis/pyxis.conf /etc/slurm/plugstack.conf
$ sudo systemctl restart slurmd
```